

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2020

Tập luận văn đội tuyển quốc gia Olympic Tin học

China Computer Federation, 2020

Nguồn gốc: Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2020

IOI China candidate team papers / National training team 2020 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2020. Tập được tạo bằng XeTeX và có lớp văn bản Unicode đọc được. Bản LaTeX này chuyển ngữ phần nhận diện tập sách, toàn bộ mục lục, và bài đầu tiên của Jiang Mingrun về tối ưu các bài toán chia khối cổ điển bằng fractional cascading.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2020*. Tập PDF gồm 196 trang và được tạo bằng XeTeX. Trang bìa không ghi riêng tên huấn luyện viên trong phần văn bản trích xuất được.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
3	Bàn về tối ưu hóa bài toán chia khối cổ điển bằng fractional cascading	Jiang Mingrun
12	Bàn về cây thống trị và ứng dụng	Chen Sunli
22	Báo cáo ra đề <i>Ghép số</i>	Jiang Xunchi
32	Báo cáo ra đề <i>Khối liên thông nhỏ nhất</i>	Pan Junyue
42	Giới thiệu đơn giản về nguyên lý chuyển vị	Chen Yu
52	Bàn về duy trì động giá trị cực trị của hàm số	Li Baitian
63	Báo cáo ra đề <i>Trò chơi trên đồ thị</i>	Zuo Junchi
74	Một loại DFT không bình thường	Zhou Yuyang
87	Bài toán rừng hướng vào nhỏ nhất	Zhang Zheyu
102	Bàn về suffix automaton nén	Xu Yixuan
117	Bàn về nimber và thuật toán đa thức	Luo Yuxiang
133	Các bài toán liên quan tới định thức trong thi tin học	Wang Zhanpeng
151	Thuật toán hiệu quả để tính diện tích vùng ghép trên mặt cầu	Zhou Renfei
173	Bàn về ứng dụng đơn giản của lý thuyết nhóm trong thi tin học	Yu Haoxiang

3 Bàn về tối ưu hóa bài toán chia khối cổ điển bằng fractional cascading

Jiang Mingrun, Trường Trung học số 7 Thành Đô

Tóm tắt

Fractional cascading là một kỹ thuật tăng tốc tìm kiếm nhị phân một giá trị trong nhiều dãy số được tổ chức theo một cấu trúc nhất định. Bài viết này giới thiệu cách dùng thuật toán đó để tối ưu một bài toán chia khối cổ điển, đồng thời trình bày một vài ứng dụng của nó.

Lời nói đầu

Thuật toán fractional cascading đã ra đời hơn 20 năm, nhưng vẫn chưa được dùng rộng rãi trong cộng đồng thi thuật toán. Bài viết này thử dùng thuật toán đó để tối ưu bài toán chia khối cổ điển, với hy vọng đưa fractional cascading vào thi tin học. Mục 1 giới thiệu fractional cascading và ý nghĩa của nó; mục 2 giới thiệu bài toán chia khối cổ điển và các cách giải thường dùng; mục 3 dùng fractional cascading để tối ưu bài toán chia khối cổ điển, so sánh với các thuật toán truyền thống, đồng thời mở rộng bài toán cổ điển.

3.1 Fractional cascading

Mục này giới thiệu thuật toán fractional cascading. Ta sẽ dùng một ví dụ cụ thể để giải thích ý nghĩa của thuật toán.

3.1.1 Giới thiệu

Trong khoa học máy tính, fractional cascading là một kỹ thuật tăng tốc tìm kiếm nhị phân một số trong nhiều dãy số được tổ chức theo một cấu trúc nhất định: lần tìm kiếm đầu tiên là tìm kiếm nhị phân thông thường với độ phức tạp $O(\log n)$, còn các lần tìm kiếm nhị phân phía sau sẽ nhanh hơn lần đầu. Fractional cascading được Chazelle và Guibas đề xuất lần đầu trong hai bài báo năm 1986 [1, 2]. Đúng như tên gọi, “fractional” mang nghĩa rời rạc, nhỏ lẻ; “cascade” là một ẩn dụ chỉ sự xếp tầng như các thác nước nhỏ. Trong bài gốc, Chazelle và Guibas đưa ra một hình minh họa, ở đây gọi là Hình 1, mô tả rất trực quan ý nghĩa sâu hơn của tên gọi này: giống như thác nước từ cao xuống thấp, dần tách thành các nhánh nhỏ hơn rồi phủ chồng lên nhau theo từng tầng [3, 4].

Hình 1: hình minh họa do Chazelle và Guibas đưa ra trong bài gốc.

3.1.2 Minh họa bằng ví dụ

Ta sẽ mô tả thuật toán này dưới dạng một bài toán ví dụ.

Xét bài toán sau: cho k dãy có thứ tự với tổng độ dài là n , dãy thứ i ký hiệu là L_i . Cần xây dựng một cấu trúc dữ liệu hỗ trợ tìm kiếm nhị phân cùng một số q trong từng dãy trong k dãy, tức tìm vị trí của phần tử đầu tiên lớn hơn hoặc bằng q . Ví dụ dưới đây có $k = 4, n = 17$:

$$L_1 = 24, 64, 65, 80, 93,$$

$$L_2 = 23, 25, 26,$$

$$L_3 = 13, 44, 62, 66,$$

$$L_4 = 11, 35, 46, 79, 81.$$

Cách đơn giản nhất là lưu riêng từng dãy. Khi đó ta chỉ cần độ phức tạp không gian $O(n)$. Nhưng khi truy vấn, ta phải tìm kiếm nhị phân riêng trên từng dãy; trong trường hợp xấu nhất, khi k dãy có độ dài bằng nhau, độ phức tạp thời gian là $O(k \log(n/k))$.

Cách thứ hai hỗ trợ truy vấn nhanh hơn nhưng tốn nhiều không gian hơn: ta gộp k dãy này thành một dãy lớn L , và với mỗi phần tử trong L lưu kết quả tìm kiếm nhị phân của nó trong k dãy ban đầu. Nếu mỗi phần tử trong dãy sau khi gộp được viết là $x[a, b, c, d]$, trong đó x là giá trị, còn a, b, c, d là vị trí của phần tử kế tiếp tương ứng của x trong các dãy ban đầu, đánh số từ 0, thì:

$$\begin{aligned} L = & 11[0, 0, 0, 0], 13[0, 0, 0, 1], 23[0, 0, 1, 1], 24[0, 1, 1, 1], 25[1, 1, 1, 1], \\ & 26[1, 2, 1, 1], 35[1, 3, 1, 1], 44[1, 3, 1, 2], 46[1, 3, 2, 2], 62[1, 3, 2, 3], \\ & 64[1, 3, 3, 3], 65[2, 3, 3, 3], 66[3, 3, 3, 3], 79[3, 3, 4, 3], 80[3, 3, 4, 4], \\ & 81[4, 3, 4, 4], 93[4, 3, 4, 5]. \end{aligned}$$

Cách này đạt độ phức tạp truy vấn $O(k + \log n)$: chỉ cần tìm kiếm nhị phân trực tiếp trong L , rồi trả về thông tin đã lưu trong phần tử x . Tuy nhiên, nó cần không gian $O(nk)$.

Fractional cascading đồng thời đạt độ phức tạp thời gian và không gian tối ưu cho cùng bài toán: truy vấn $O(k + \log n)$, không gian $O(n)$. Ta xây dựng k dãy có thứ tự mới, dãy thứ i ký hiệu là M_i . Trong đó, dãy cuối cùng M_k giống L_k . Với mỗi dãy phía trước, M_i được tạo bằng cách trộn L_i với dãy con lấy mẫu cách một phần tử của M_{i+1} , bắt đầu từ phần tử thứ hai; với mỗi phần tử trong M_i , ta lưu kết quả tìm kiếm nhị phân của nó trong L_i và M_{i+1} . Với 4 dãy ở trên, ta có:

$$\begin{aligned} M_1 = & 24[0, 1], 25[1, 1], 35[1, 3], 64[1, 5], 65[2, 5], 79[3, 5], 80[3, 6], 93[4, 6], \\ M_2 = & 23[0, 1], 25[1, 1], 26[2, 1], 35[3, 1], 62[3, 3], 79[3, 5], \\ M_3 = & 13[0, 1], 35[1, 1], 44[1, 2], 62[2, 3], 66[3, 3], 79[4, 3], \\ M_4 = & 11[0, 0], 35[1, 0], 46[2, 0], 79[3, 0], 81[4, 0]. \end{aligned}$$

Nếu muốn truy vấn $q = 50$, trước hết ta tìm kiếm nhị phân trong M_1 và nhận được $64[1, 5]$. Số 1 nghĩa là kết quả tìm kiếm nhị phân trong L_1 là $L_1[1] = 64$; số 5 nghĩa là kết quả tìm kiếm nhị phân trong M_2 nằm xấp xỉ ở chỉ số 5. Chính xác hơn, nó là phần tử $79[3, 5]$ ở chỉ số 5 hoặc phần tử $62[3, 3]$ ngay trước đó. So sánh q với 62, ta biết kết quả đúng là $62[3, 3]$. Bằng cùng phương pháp, ta có thể nhận được kết quả tìm kiếm nhị phân của q trong L_2, M_3, L_3 , và M_4 .

Nói tổng quát hơn, với bất kỳ cấu trúc như vậy, ta có thể tìm kiếm nhị phân trước trong M_1 . Sau đó với mọi i , từ vị trí của q trong M_i ta định vị được vị trí của q trong L_i và trong M_{i+1} . Vị trí trong M_{i+1} mà kết quả truy vấn ở M_i trở tới hoặc chính là kết quả đúng của truy vấn trong M_{i+1} , hoặc chỉ lệch một vị trí, nên một phép so sánh là đủ để xác định. Tổng độ phức tạp thời gian là $O(k + \log n)$.

Trong ví dụ trên, tổng độ dài của bốn dãy mới là 25, không vượt quá $2n$. Nói chung, độ dài của M_i không vượt quá

$$|L_i| + \frac{1}{2}|L_{i+1}| + \frac{1}{4}|L_{i+2}| + \dots.$$

Tổng độ dài của mọi M_i không vượt quá

$$\sum |M_i| \leq \sum |L_i| \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots\right) = 2n = O(n).$$

3.1.3 Trường hợp tổng quát

Xét một đồ thị có hướng không chu trình. Bậc vào và bậc ra của mỗi đỉnh đều không vượt quá một hằng số d , và trên mỗi đỉnh có lưu một dãy có thứ tự L_u . Một truy vấn cần tìm kiếm nhị phân một số q trong các dãy L_u được lưu trên mọi đỉnh u của một đường đi. Trong ví dụ phía trên, đồ thị có hướng đã cho là một dây chuyền độ dài 4.

Với mỗi đỉnh u , ta xây dựng một dãy mới M_u . Dãy M_u được tạo bằng cách trộn L_u với các dãy thu được khi lấy mẫu đều theo một tỉ lệ nhất định từ M_v của mọi hậu duệ trực tiếp v của u ; nếu muốn đạt không gian $O(n)$, tỉ lệ lấy mẫu phải nhỏ hơn $1/d$. Với mỗi phần tử, ta lưu kết quả tìm kiếm nhị phân của nó trong L_u và trong từng M_v . Trong ví dụ phía trên, $d = 1$ và tỉ lệ lấy mẫu là $1/2$.

Với một truy vấn, trước hết ta thực hiện một lần tìm kiếm nhị phân $O(\log n)$ trong dãy M_s được xây dựng ở điểm đầu s của đường đi. Giả sử ta đã biết kết quả tìm kiếm nhị phân của q trong M_u , và cần tìm kết quả tìm kiếm nhị phân của q trong M_v với một hậu duệ trực tiếp v của u . Nếu tỉ lệ lấy mẫu khi xây dựng cấu trúc dữ liệu là $1/r$, thì khoảng cách giữa vị trí trong M_v mà kết quả tìm kiếm nhị phân trong M_u trở tới và vị trí đúng trong M_v sẽ không vượt quá r . Vì r là hằng số, ta có thể tìm kết quả đúng trong M_v với độ phức tạp thời gian $O(1)$.

Nếu độ dài đường đi cần truy vấn là k , độ phức tạp thời gian của một truy vấn là $O(k + \log n)$.

Ngoài ra, fractional cascading có thể hỗ trợ chèn một phần tử vào một dãy trong $O(\log n)$, nhưng độ phức tạp truy vấn sẽ trở thành $O(\log n + k \log \log n)$. Điều này không liên quan nhiều tới bài viết này nên được lược bỏ.

3.2 Bài toán chia khối cổ điển

Mục này giới thiệu một bài toán chia khối kinh điển trong OI và cách giải truyền thống của nó. Ở mục sau ta sẽ thảo luận cách dùng fractional cascading để tối ưu bài toán này.

3.2.1 Mô tả bài toán

Cho một dãy độ dài n và n thao tác. Có hai loại thao tác:

1. Thao tác sửa: cho l, r, x ; với mọi $l \leq i \leq r$, tăng a_i thêm x .
2. Thao tác hỏi: cho l, r, x ; hỏi trong mọi $l \leq i \leq r$, có bao nhiêu chỉ số i thỏa $a_i \leq x$.

3.2.2 Cách giải thường dùng, thuật toán 1

Chia dãy thành các khối, mỗi khối gồm B phần tử liên tiếp, tổng cộng $O(n/B)$ khối. Trong mỗi khối, lưu kết quả sắp xếp của mọi phần tử trong khối. Như vậy với một đoạn $[l, r]$, nhiều nhất chỉ có 2 khối bị phủ không hoàn toàn; các khối còn lại hoặc được phủ hoàn toàn, hoặc không bị phủ.

Với thao tác sửa, các khối bị phủ không hoàn toàn cần được xây dựng lại trực tiếp. Lúc này phải tính lại kết quả sắp xếp. Nếu sắp xếp thô thì độ phức tạp là $O(B \log B)$; nhưng chú ý rằng với hai số đều không bị sửa hoặc hai số đều bị sửa, quan hệ lớn nhỏ giữa chúng không đổi. Vì vậy, nếu khi sắp xếp ta ghi lại vị trí của phần tử trước khi sắp xếp, ta có thể sắp riêng các số không bị sửa và các số bị sửa trong $O(B)$, rồi trộn lại, nên độ phức tạp thời gian là $O(B)$. Với khối được phủ hoàn toàn, ta có thể ghi một thể lười biểu thị rằng mọi phần tử trong khối đều được cộng cùng một số. Độ phức tạp cho mỗi khối là $O(1)$, tổng cộng có $O(n/B)$ khối, nên tổng độ phức tạp thời gian là $O(n/B)$.

Với thao tác hỏi, các khối bị phủ không hoàn toàn có thể được duyệt trực tiếp qua mọi phần tử, độ phức tạp $O(B)$. Với khối được phủ hoàn toàn, ta cần tìm kiếm nhị phân trên mảng đã sắp xếp, độ phức tạp cho mỗi khối là $O(\log B)$, tổng độ phức tạp thời gian là $O(n \log B/B)$.

Tóm lại, độ phức tạp thời gian cho một thao tác là $O(B + n \log B/B)$; tổng độ phức tạp thời gian là $O(nB + n^2 \log B/B)$. Khi lấy $B = \sqrt{n \log n}$, độ phức tạp thời gian là tối ưu và bằng $O(n\sqrt{n \log n})$.

3.2.3 Thuật toán offline, thuật toán 2

Mục này thảo luận thuật toán offline.

Trong bài này, thuật toán ở mục này chỉ dùng được với phạm vi đầu vào sau: mọi số trong đầu vào đều là số nguyên có trị tuyệt đối không vượt quá 2^ω , và $\omega = O(\log n)$.

Chú ý rằng nút thắt của thuật toán ở mục trước nằm ở tìm kiếm nhị phân, nên mục này xét cách tối ưu phần đó.

Tuy nhiên, tối ưu một lần tìm kiếm nhị phân đơn lẻ là việc khó; có thể dùng các cấu trúc dữ liệu như y-fast tree, nhưng không thực dụng. Do đó, để tối ưu thuật toán, ta cần tối ưu tổng độ phức tạp thời gian của nhiều lần tìm kiếm nhị phân. Có hai hướng: hướng thứ nhất là tối ưu nhiều truy vấn tìm kiếm nhị phân trong một khối; hướng thứ hai là tối ưu một truy vấn tìm kiếm nhị phân trên nhiều khối. Mục này thảo luận hướng thứ nhất, còn hướng thứ hai sẽ được thảo luận ở mục sau.

Xét việc lưu trước mọi truy vấn và xử lý từng khối. Khi xây dựng lại một khối, ta tính kết quả của mọi lần tìm kiếm nhị phân trước đó trên khối này. Khi đó bài toán chuyển thành thực hiện Q lần tìm kiếm nhị phân trong một dãy độ dài B .

Chú ý rằng nếu các số được tìm kiếm nhị phân tăng đơn điệu, ta có thể tận dụng tính đơn điệu để đạt độ phức tạp $O(B + Q)$. Vì vậy nếu có thể sắp xếp nhanh Q số, ta có thể nhanh chóng tính được kết quả của Q lần tìm kiếm nhị phân.

Khi $\omega = O(\log B)$, có thể dùng sắp xếp cơ số với cơ số B , độ phức tạp thời gian là

$$O\left(\frac{(B + Q)\omega}{\log B}\right) = O(B + Q).$$

Như vậy, độ phức tạp thời gian của bài toán chia khối ban đầu có thể được tối ưu thành $O(nB + n^2/B)$; lấy $B = \sqrt{n}$ là tối ưu, cho $O(n\sqrt{n})$.

3.3 Thuật toán dựa trên fractional cascading

Mục trước đã giới thiệu bài toán chia khối cổ điển trong OI và các cách giải truyền thống. Mục này thảo luận cách dùng fractional cascading để tối ưu bài toán đó.

Cách dùng sắp xếp cơ số có thể đạt độ phức tạp thời gian $O(n\sqrt{n})$, nhưng nó không còn dùng được nếu miền giá trị không phải số nguyên, hoặc nếu bài toán bị buộc phải online, nghĩa là trước khi đọc một thao tác mới thì phải tính xong kết quả của mọi truy vấn trước đó, nên không thể lưu trước toàn bộ truy vấn.

Một lần tìm kiếm nhị phân riêng lẻ rất khó tối ưu. Nhưng cần chú ý rằng thứ ta cần không phải một lần tìm kiếm nhị phân riêng lẻ, mà là tìm kiếm nhị phân cùng một số trên $O(n/B)$ dãy. Vì vậy, có thể xét dùng fractional cascading. Tuy nhiên, fractional cascading không hỗ trợ sửa trực tiếp, nên cần xử lý thêm.

3.3.1 Cách làm $O(n\sqrt{n} \log \log n)$, thuật toán 3

Cai Chengze của Đại học Thanh Hoa đã nghĩ ra thuật toán có độ phức tạp thời gian $O(n\sqrt{n} \log \log n)$ trước tác giả, nhưng chưa công bố chi tiết. Cách làm được mô tả trong mục này là một cách khả dĩ mà tác giả suy đoán từ một số trao đổi liên quan; không rõ có trùng với cách làm của Cai hay không. Dù thế nào, xin cảm ơn Cai Chengze.

Trong thuật toán này, cách xây dựng fractional cascading là: trong mỗi khối, lấy mẫu đều $1/D$ số phần tử để xây dựng fractional cascading. Khi tìm kiếm nhị phân, nhờ fractional cascading ta định vị được một vị trí lệch không quá D , do đó trong mỗi khối riêng lẻ đạt độ phức tạp thời gian $O(\log D)$. Dùng cách này, cứ mỗi D khối liên tiếp xây dựng một nhóm fractional cascading; tổng cộng có $O(n/(BD))$ nhóm.

Khi thực hiện thao tác sửa, nhiều nhất chỉ có 2 nhóm fractional cascading bị phá vỡ cấu trúc; các nhóm khác nhiều nhất chỉ bị cộng cùng một số vào mọi phần tử, điều này không thay đổi cấu trúc fractional cascading. Vì mỗi khối chỉ lấy $O(B/D)$ phần tử để xây dựng fractional cascading, và mỗi nhóm fractional cascading chỉ được xây dựng giữa D khối, nên độ phức tạp thời gian để xây dựng lại một nhóm là $O(B)$.

Với thao tác hỏi, cần thực hiện một truy vấn trong mỗi nhóm fractional cascading. Độ phức tạp truy vấn trong một nhóm là $O(D + \log B)$; có tổng cộng $O(n/(BD))$ nhóm, nên tổng độ phức tạp là $O(n/B + n \log B/(BD))$. Ngoài ra, với mỗi khối còn cần một lần tìm kiếm nhị phân $O(\log D)$, tổng độ phức tạp là $O(n \log D/B)$. Ta cũng phải duyệt $O(B)$ phần tử trong các khối bị phủ không hoàn toàn.

Tóm lại, độ phức tạp thời gian của một thao tác có thể được tối ưu thành

$$O\left(B + \frac{n \log D}{B} + \frac{n \log B}{BD}\right),$$

tổng độ phức tạp thời gian là

$$O\left(nB + \frac{n^2 \log D}{B} + \frac{n^2 \log B}{BD}\right).$$

Lấy $B = \sqrt{n \log \log n}$, $D = \log n$, độ phức tạp thời gian là $O(n\sqrt{n \log \log n})$.

3.3.2 Cách làm $O(n\sqrt{n})$, thuật toán 4

Nếu dùng cách gộp ở mục 1.2 thì đã khó tối ưu tiếp. Tiếp theo, mục này bắt đầu từ một cách gộp khác để tối ưu bài toán thêm một bước.

Xây dựng một cây đoạn có n/B lá; các lá của cây đoạn lần lượt lưu dãy đã sắp xếp được duy trì trong mỗi khối. Dãy lưu ở đỉnh trong được tạo bằng cách trộn các dãy thu được khi lấy mẫu đều theo một tỉ lệ nhất định từ dãy lưu ở hai con, và ghi lại với mỗi phần tử kết quả tìm kiếm nhị phân của nó trong hai dãy lưu ở hai con. Nói cách khác, trong mục 1.3, cho mỗi đỉnh trong của cây đoạn nối tới hai con của nó để tạo đồ thị có hướng không chu trình; L_u ở lá là dãy được duy trì trong mỗi khối, còn L_u ở đỉnh trong là dãy rỗng, từ đó nhận được cấu trúc fractional cascading.

Khi thực hiện thao tác sửa, theo tính chất của cây đoạn, chỉ có $O(\log(n/B))$ đỉnh có đoạn tương ứng bị phủ không hoàn toàn. Nếu đoạn ứng với một đỉnh được phủ hoàn toàn, mọi phần tử trong cây con của đỉnh đó chỉ bị cộng thêm cùng một số, còn cấu trúc không đổi. Vì vậy chỉ có $O(\log(n/B))$ đỉnh cần tính lại dãy lưu trữ. Khi tỉ lệ lấy mẫu là $1/2$, độ dài dãy lưu ở mỗi đỉnh đều là $O(B)$, tổng độ phức tạp thời gian là $O(B \log(n/B))$, chưa đủ tốt. Nhưng nếu dùng tỉ lệ nhỏ hơn, chẳng hạn $1/3$, thì ở mỗi tầng của cây đoạn, độ dài dãy lưu ở mỗi đỉnh bằng $2/3$ so với tầng dưới; theo tính chất của cây đoạn, ở mỗi tầng chỉ có $O(1)$ đỉnh bị phủ không hoàn toàn. Vì vậy tổng độ phức tạp thời gian không vượt quá

$$O\left(B\left(1 + \frac{2}{3} + \frac{4}{9} + \frac{8}{27} + \dots\right)\right) = O(B).$$

Với thao tác hỏi, tương tự mục 1.3, ta có thể tìm kiếm nhị phân trước trong gốc, rồi từ kết quả tìm kiếm nhị phân trong một đỉnh xác định trong $O(1)$ kết quả tìm kiếm nhị phân ở hai con. Tổng độ phức tạp thời gian là $O(n/B + \log n)$. Ngoài ra, ta vẫn cần duyệt $O(B)$ phần tử trong các khối bị phủ không hoàn toàn.

Tóm lại, độ phức tạp thời gian của một thao tác là $O(B + n/B + \log n)$; tổng độ phức tạp thời gian là $O(nB + n^2/B + n \log n)$. Lấy $B = \sqrt{n}$, độ phức tạp thời gian là tối ưu và bằng $O(n\sqrt{n})$.

3.3.3 Tổng kết thuật toán

Tổng hợp lại, độ phức tạp thời gian của các thuật toán được nêu trong Bảng 1, trong đó thuật toán 1 là thuật toán truyền thống. Thuật toán 2 cũng đạt mục tiêu tối ưu, nhưng chỉ phù hợp với một phần phạm vi

đầu vào. Thuật toán 3 và thuật toán 4 đều là các thuật toán tối ưu bằng fractional cascading, không bị hạn chế khi sử dụng; trong các thuật toán đã biết, thuật toán 4 là tối ưu nhất.

Thuật toán	Độ phức tạp thời gian	Ghi chú
Thuật toán 1	$O(n\sqrt{n \log n})$	Chỉ phù hợp với một phần phạm vi đầu vào
Thuật toán 2	$O(n\sqrt{n})$	
Thuật toán 3	$O(n\sqrt{n \log \log n})$	
Thuật toán 4	$O(n\sqrt{n})$	

Bảng 1: So sánh các thuật toán

3.3.4 Mở rộng ứng dụng thuật toán

Fractional cascading không chỉ có thể áp dụng cho bài toán cổ điển này, mà còn có thể mở rộng sang một số bài toán khác.

Bài toán 1. Cho một dãy độ dài n và n thao tác. Có hai loại thao tác:

1. Thao tác sửa: cho l, r, x ; với mọi $l \leq i \leq r$, tăng a_i thêm x .
2. Thao tác hỏi: cho l, r, x ; hỏi đoạn $[l, r]$ có bao nhiêu đoạn con $[l', r']$ thỏa mọi số trong $[l', r']$ đều không vượt quá x .¹

Chú ý rằng với hai đoạn $[l, mid]$ và $[mid + 1, r]$, nếu ta biết đáp án của chúng, ở đây “đáp án” bao gồm đáp án của thao tác hỏi, vị trí của số đầu tiên lớn hơn x trong đoạn, và vị trí của số cuối cùng lớn hơn x trong đoạn, thì ta có thể tính đáp án của đoạn $[l, r]$.

Chia mỗi B số liên tiếp thành một khối, tổng cộng $O(n/B)$ khối. Với một truy vấn, ta có thể tách đoạn $[l, r]$ thành $O(n/B)$ đoạn có độ dài không vượt quá B , trong đó chỉ có $O(1)$ đoạn không phải là nguyên một khối. Với đoạn không phải nguyên một khối, có thể tách tiếp thành $O(B)$ đoạn độ dài 1 để xử lý. Với một khối nguyên vẹn, ta tính dãy đã sắp xếp của mọi phần tử trong khối, và tính đáp án của khối khi từng phần tử trong khối được dùng làm x . Phần sau có thể được tính bằng cách sau:

Xét việc xử lý theo thứ tự tăng dần. Đưa mọi số lớn hơn x vào một danh sách liên kết theo thứ tự vị trí xuất hiện; để tiện xử lý, có thể thêm nút đặc biệt ở hai đầu danh sách. Khi x tăng dần, các phần tử trong danh sách sẽ dần bị xóa. Khi tính đáp án của khối, vị trí số đầu tiên lớn hơn x và vị trí số cuối cùng lớn hơn x trong đoạn có thể được lấy trực tiếp qua con trỏ của các nút đặc biệt. Khi tính đáp án thao tác hỏi, cần xét ảnh hưởng của việc xóa một nút lên đáp án: rõ ràng mọi đoạn chứa nút này nhưng không chứa tiền nhiệm và hậu nhiệm của nút này sẽ được cộng vào đáp án, và không có ảnh hưởng nào khác. Bước này có độ phức tạp thời gian $O(B)$, nên không làm tăng độ phức tạp thời gian xây dựng lại khối.

Sau khi tính được phần này, phần còn lại cơ bản giống cách giải bài toán chia khối cổ điển ở mục 2, và có thể được tối ưu bằng phương pháp ở mục 3.2.

Bài toán 2. Cho một dãy độ dài n và n thao tác. Có hai loại thao tác:

1. Thao tác sửa: cho l, r, x ; với mọi $l \leq i \leq r$, tăng a_i thêm x .
2. Thao tác hỏi: cho l, r, x ; hỏi trong mọi đoạn con của đoạn $[l, r]$, tổng các phần tử trong đoạn con lớn nhất là bao nhiêu.

¹Bài toán này được chỉnh sửa từ Comet OJ Contest #7 F: https://cometoj.com/contest/52/problem/F?problem_id=2426.

Trước hết trình bày cách làm của lời giải chính thức.

Tương tự bài toán trước, với hai đoạn $[l, mid]$ và $[mid + 1, r]$, nếu ta biết đáp án của chúng, ở đây “đáp án” bao gồm tổng đoạn con lớn nhất trong đoạn, tức đáp án của thao tác hỏi, tổng tiền tố lớn nhất trong đoạn, và tổng hậu tố lớn nhất trong đoạn, thì ta có thể tính đáp án của đoạn $[l, r]$.

Vấn xét chia mỗi B số liên tiếp thành một khối, tổng cộng $O(n/B)$ khối. Điểm mấu chốt là xử lý trường hợp một khối nguyên vẹn.

Để xử lý một khối nguyên vẹn, ta cần duy trì đáp án khi mọi số trong cả khối được cộng cùng một số. Trước hết xử lý tình huống này.

Xét dùng cây đoạn để duy trì. Với một đỉnh, cần duy trì đáp án của đoạn mà đỉnh đó đại diện khi mọi số trong đoạn được cộng cùng một số x . Không khó chứng minh đây là một hàm tuyến tính từng đoạn theo x , và số đoạn cùng bậc với độ dài của đoạn. Thông tin duy trì ở một đỉnh có thể được gộp từ hai con của nó; phép gộp chỉ cần cộng hai hàm từng đoạn hoặc lấy max, có thể hoàn thành trong $O(len)$, trong đó len là số đoạn của hàm từng đoạn, cùng bậc với độ dài đoạn. Như vậy, độ phức tạp thời gian xây dựng cây đoạn là tổng độ dài các đoạn tương ứng với mọi đỉnh trong cây đoạn, tức $O(B \log B)$.

Nhưng với bài toán ban đầu, làm như vậy sẽ gặp vài vấn đề. Vấn đề thứ nhất là khi thực hiện thao tác sửa, cần xây dựng lại các khối bị phủ không hoàn toàn. Nếu xây thô lại toàn bộ cây đoạn thì độ phức tạp thời gian là $O(B \log B)$, chưa đủ tốt.

Chú ý rằng khi xây dựng lại một khối, thao tác sửa trên khối đó chỉ là cộng đoạn. Theo tính chất của cây đoạn, ở mỗi tầng chỉ có $O(1)$ đỉnh bị phủ không hoàn toàn, còn sửa trên đỉnh được phủ hoàn toàn chỉ là tịnh tiến hàm từng đoạn, có thể dùng thẻ lười để duy trì. Vì vậy, ở mỗi tầng chỉ có $O(1)$ đỉnh cần tính lại thông tin, độ phức tạp thời gian là

$$O\left(B + \frac{B}{2} + \frac{B}{4} + \frac{B}{8} + \dots\right) = O(B).$$

Vấn đề thứ hai là khi hỏi đáp án của một khối, ta cần tìm kiếm nhị phân để biết đáp án nằm ở đoạn nào của hàm từng đoạn. Nếu tìm kiếm nhị phân trực tiếp, độ phức tạp của một thao tác là $O(B + n \log B/B)$, chưa đủ tốt. Cách trong lời giải chính thức là tối ưu tương tự mục 2.3, tức dùng sắp xếp cơ số rồi tận dụng tính đơn điệu. Độ phức tạp thời gian là $O(n\sqrt{n})$. Do xử lý từng khối, ta có thể chỉ xây dựng cây đoạn của khối đang được xử lý, nên độ phức tạp không gian là $O(B \log B + n) = O(n)$, chứ không phải $O(n \log n)$ như khi xây tất cả cây đoạn của mọi khối.

Thực ra ta có thể dùng phương pháp fractional cascading của thuật toán 4 ở mục 3.2, trong đó dãy được lưu ở các lá là dãy gồm các điểm phân chia của hàm từng đoạn. Với khối được phủ hoàn toàn, thao tác sửa chỉ tịnh tiến hàm từng đoạn, tức cộng cùng một số vào mọi điểm phân chia. Vì vậy nếu đoạn tương ứng với một đỉnh trong cây đoạn được phủ hoàn toàn, mọi dãy mà các đỉnh trong cây con của nó duy trì cũng chỉ bị cộng cùng một số vào mọi phần tử, không làm thay đổi cấu trúc. Do đó cách làm ở mục 3.2 vẫn áp dụng được, với độ phức tạp thời gian $O(n\sqrt{n})$ và độ phức tạp không gian $O(n \log n)$.

3.4 Kết luận

Fractional cascading tuy đã có hơn 20 năm lịch sử và là một thuật toán phổ biến, nhưng hiện vẫn chưa được ứng dụng rộng rãi trong thi tin học. Bài viết này giới thiệu fractional cascading, so sánh thuật toán này với các thuật toán khác, và áp dụng nó vào một số bài toán trong thi tin học, đặc biệt là tối ưu thuật toán khi giải dạng bài chia khối cổ điển này. Hy vọng bài viết có ích cho mọi người.

Tuy nhiên, chưa có nhiều người nghiên cứu lĩnh vực này, và do trình độ tác giả còn hạn chế, vẫn còn nhiều vấn đề chưa được giải quyết. Mong bài viết này có thể gợi mở thêm thảo luận và thu hút nhiều đồng nghiệp hơn nghiên cứu thuật toán này.

Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn huấn luyện viên Gao Wenyuan của đội tuyển quốc gia đã hướng dẫn. Xin cảm ơn các thầy Lin Yang, Zhang Junliang, Hu Fan, Li Zhiwu, và Lin Hong đã bồi dưỡng và chỉ dạy tôi. Xin cảm ơn các anh Cai Chengze, Li Xinlong, v.v. đã cho tôi cảm hứng trong các cuộc thảo luận. Xin cảm ơn các anh Wang Xiuhan, Wang Siqi, Yuan Fangzhou, v.v. đã chỉ dẫn. Xin cảm ơn toàn thể các bạn lớp tin học khóa 2018 của Trường Trung học số 7 Thành Đô đã đồng hành trong hai năm. Xin cảm ơn cha mẹ đã thấu hiểu và ủng hộ tôi.

Tài liệu tham khảo

1. Chazelle, Bernard; Guibas, Leonidas J. *Fractional cascading: I. A data structuring technique*. 1986.
2. Chazelle, Bernard; Guibas, Leonidas J. *Fractional cascading: II. Applications*. 1986.
3. Wikipedia contributors. *Fractional cascading*. Wikipedia. https://en.wikipedia.org/wiki/Fractional_cascading.
4. Blog CSDN của Baimafujinji: blog.csdn.net/baimafujinji/.../52956605.